



Aula 5 — Switch e Case em C

Até aqui, já aprendemos a fazer o programa tomar decisões usando `if`, `else if` e `else`.

Mas imagine que estamos criando um programa com um menu.

Por exemplo:

```
1 - Ver saldo
2 - Fazer levantamento
3 - Depositar dinheiro
4 - Sair
```

Podemos fazer isso usando vários `if` e `else if`.

Mas existe uma maneira mais organizada de trabalhar quando temos várias opções baseadas em um único valor.

É aí que entra o `switch`.

1. O que é switch?

O `switch` permite que o programa escolha uma opção com base no valor de uma variável.

Pensa em um controle remoto.

Você aperta:

```
1 → Canal 1
2 → Canal 2
3 → Canal 3
```

O programa verifica o número que você escolheu e executa a ação correspondente.

Em C, podemos fazer isso com:

```
switch (variavel) {

    case valor:
        // código
        break;

}
```

O `switch` significa basicamente:

"Vou verificar o valor desta variável e decidir qual opção executar."

2. O que é case?

O `case` representa cada uma das opções possíveis.

Por exemplo:

```
#include <stdio.h>

int main() {

    int opcao;

    printf("Digite uma opcao: ");
    scanf("%d", &opcao);

    switch (opcao) {

        case 1:
            printf("Voce escolheu a opcao 1.");
            break;

        case 2:
            printf("Voce escolheu a opcao 2.");
            break;

        case 3:
            printf("Voce escolheu a opcao 3.");
            break;

    }

    return 0;
}
```

Se o utilizador digitar:

2

o programa encontra:

`case 2:`

e executa:

```
printf("Voce escolheu a opcao 2.");
```

3. Para que serve o break?

Aqui aparece uma palavra muito importante:

```
break;
```

O `break` diz ao programa:

"Já encontrei a opção que quero. Pode sair do `switch`."

Por exemplo:

```
case 1:  
    printf("Opcao 1");  
    break;
```

Sem o `break`, o programa pode continuar executando os próximos `case`.

No início, basta lembrar:

`break` = parar o `switch` aqui.

4. O default

E se o utilizador digitar uma opção que não existe?

Por exemplo:

```
1  
2  
3
```

Mas ele digita:

```
7
```

Podemos usar `default`.

O `default` funciona como o nosso "caso contrário".

```

#include <stdio.h>

int main() {

    int opcao;

    printf("Digite uma opcao: ");
    scanf("%d", &opcao);

    switch (opcao) {

        case 1:
            printf("Opcao 1 escolhida.");
            break;

        case 2:
            printf("Opcao 2 escolhida.");
            break;

        case 3:
            printf("Opcao 3 escolhida.");
            break;

        default:
            printf("Opcao invalida.");
    }

    return 0;
}

```

Se o utilizador digitar :

Opcao invalida.

Isso é muito útil para validar menus.

5. Criando nosso primeiro menu

Agora vamos fazer algo mais interessante.

Imagine que temos um pequeno programa com três opções:

=== MENU ===

1 - Ver mensagem

- 2 - Mostrar idade
- 3 - Sair

Podemos fazer:

```
#include <stdio.h>

int main() {

    int opcao;
    int idade = 18;

    printf("=== MENU ===\n");
    printf("1 - Ver mensagem\n");
    printf("2 - Mostrar idade\n");
    printf("3 - Sair\n");

    printf("\nEscolha uma opcao: ");
    scanf("%d", &opcao);

    switch (opcao) {

        case 1:
            printf("Bem-vindo ao programa!");
            break;

        case 2:
            printf("Sua idade e: %d", idade);
            break;

        case 3:
            printf("Programa encerrado.");
            break;

        default:
            printf("Opcao invalida.");
    }

    return 0;
}
```

Agora o nosso programa já parece um pequeno sistema.

6. Switch ou if?

Você pode estar pensando:



“Mas eu já consigo fazer isso com `if`.”

E está certo.

Por exemplo, poderíamos fazer:

```
if (opcao == 1) {  
    printf("Opcao 1");  
}  
else if (opcao == 2) {  
    printf("Opcao 2");  
}  
else if (opcao == 3) {  
    printf("Opcao 3");  
}  
else {  
    printf("Opcao invalida");  
}
```

Também funciona.

Então qual é a diferença?

De forma simples:

Quando temos várias possibilidades baseadas no valor de uma mesma variável, `switch` pode deixar o código mais organizado e fácil de ler.

Por exemplo:

```
switch (opcao) {  
  
    case 1:  
        // ...  
        break;  
  
    case 2:  
        // ...  
        break;  
  
    case 3:  
        // ...  
        break;  
  
}
```

Fica bastante claro que estamos trabalhando com um menu de opções.

7. Switch com caracteres

O `switch` não precisa trabalhar apenas com números.

Também podemos utilizar `char`.

Por exemplo, imagine um programa que pergunta uma operação matemática:

```
+ → Soma
- → Subtração
* → Multiplicação
/ → Divisão
```

Podemos fazer:

```
#include <stdio.h>

int main() {

    char operador;

    printf("Digite um operador (+, -, *, /): ");
    scanf(" %c", &operador);

    switch (operador) {

        case '+':
            printf("Voce escolheu soma.");
            break;

        case '-':
            printf("Voce escolheu subtracao.");
            break;

        case '*':
            printf("Voce escolheu multiplicacao.");
            break;

        case '/':
            printf("Voce escolheu divisao.");
            break;

        default:
            printf("Operador invalido.");
    }
}
```

```
    return 0;  
}
```

Repare que os caracteres ficam entre aspas simples:

```
'+'  
'-'  
'*'  
'/'
```

Enquanto textos ficam entre aspas duplas:

```
"Olá"
```

8. Um detalhe importante no scanf

Talvez você tenha reparado nesta linha:

```
scanf(" %c", &operador);
```

Existe um espaço antes de `%c`:

```
" %c"
```

Esse espaço ajuda o `scanf()` a ignorar caracteres que ficaram no teclado, como o `Enter` pressionado anteriormente.

Por enquanto, apenas guarde essa ideia.

Mais à frente podemos entender melhor como a entrada de dados funciona.

9. Switch dentro de um loop

Agora vamos deixar o nosso programa mais interessante.

Imagine que queremos que o menu apareça novamente depois de cada operação.

Podemos utilizar um `while` junto com `switch`.

```

#include <stdio.h>

int main() {

    int opcao;

    while (opcao != 3) {

        printf("\n=== MENU ===\n");
        printf("1 - Mostrar mensagem\n");
        printf("2 - Mostrar idade\n");
        printf("3 - Sair\n");

        printf("Escolha uma opcao: ");
        scanf("%d", &opcao);

        switch (opcao) {

            case 1:
                printf("Bem-vindo!");
                break;

            case 2:
                printf("Voce tem 18 anos.");
                break;

            case 3:
                printf("Saindo do programa...");
                break;

            default:
                printf("Opcao invalida.");

        }

    }

    return 0;
}

```

Aqui temos uma combinação muito importante:

```

while
↓
mantém o menu funcionando

switch
↓
escolhe a opção

case

```

↓
executa a ação

break
↓
sai do switch

Agora já estamos começando a construir programas com menus de verdade.

10. Projeto — Calculadora simples

Vamos fazer um pequeno projeto utilizando `switch`.

O programa vai receber dois números e perguntar qual operação queremos fazer.

```
#include <stdio.h>

int main() {

    float numero1, numero2, resultado;
    char operador;

    printf("=== CALCULADORA ===\n\n");

    printf("Digite o primeiro numero: ");
    scanf("%f", &numero1);

    printf("Digite o segundo numero: ");
    scanf("%f", &numero2);

    printf("Digite a operacao (+, -, *, /): ");
    scanf(" %c", &operador);

    switch (operador) {

        case '+':
            resultado = numero1 + numero2;
            printf("Resultado: %.2f", resultado);
            break;

        case '-':
            resultado = numero1 - numero2;
            printf("Resultado: %.2f", resultado);
            break;

        case '*':
            resultado = numero1 * numero2;
            printf("Resultado: %.2f", resultado);
```

```

        break;

    case '/':

        if (numero2 == 0) {
            printf("Erro: nao e possivel dividir por zero.");
        } else {
            resultado = numero1 / numero2;
            printf("Resultado: %.2f", resultado);
        }

        break;

    default:
        printf("Operador invalido.");
}

return 0;
}

```

Aqui temos uma coisa interessante.

Estamos usando `switch` para escolher a operação:

```

+ → soma
- → subtração
* → multiplicação
/ → divisão

```

Mas dentro do `case '/'` usamos um `if`.

Isso mostra uma coisa importante:

As estruturas de programação podem trabalhar juntas.

Não precisamos escolher apenas uma ferramenta.

Podemos usar:

```

switch
+
if
+
variáveis
+
operadores

```

para resolver um problema.

11. Projeto maior — Menu de conta

Agora podemos tentar algo mais próximo de um sistema real.

Vamos criar um menu simples:

```
=== MINHA CONTA ===  
  
1 - Ver saldo  
2 - Depositar  
3 - Levantar  
4 - Sair
```

O programa terá um saldo inicial de 10.000 Kz.

```
#include <stdio.h>  
  
int main() {  
  
    int opcao;  
    float saldo = 10000;  
    float valor;  
  
    do {  
  
        printf("\n=====\\n");  
        printf("    MINHA CONTA\\n");  
        printf("=====\\n");  
  
        printf("1 - Ver saldo\\n");  
        printf("2 - Depositar\\n");  
        printf("3 - Levantar\\n");  
        printf("4 - Sair\\n");  
  
        printf("\\nEscolha uma opcao: ");  
        scanf("%d", &opcao);  
  
        switch (opcao) {  
  
            case 1:  
                printf("\\nSaldo atual: %.2f Kz\\n", saldo);  
                break;  
  
            case 2:  
  
                printf("\\nDigite o valor para depositar: ");  
                scanf("%f", &valor);
```

```

        if (valor > 0) {
            saldo = saldo + valor;
            printf("Deposito realizado com sucesso!\n");
        } else {
            printf("Valor invalido.\n");
        }

        break;

    case 3:

        printf("\nDigite o valor para levantar: ");
        scanf("%f", &valor);

        if (valor <= 0) {
            printf("Valor invalido.\n");
        }
        else if (valor > saldo) {
            printf("Saldo insuficiente.\n");
        }
        else {
            saldo = saldo - valor;
            printf("Levantamento realizado com sucesso!\n");
        }

        break;

    case 4:
        printf("\nObrigado por utilizar o sistema.\n");
        break;

    default:
        printf("\nOpcao invalida.\n");
    }

} while (opcao != 4);

return 0;
}

```

Aqui já estamos juntando praticamente tudo que aprendemos:

Variáveis

↓

guardam os dados

do while

↓

mantém o menu funcionando

switch



escolhe a opção

case



executa a ação

if / else



faz validações

Operadores



fazem cálculos e comparações

Esse tipo de exercício é importante porque começamos a perceber que programação não é aprender comandos isolados.

É aprender a combinar as ferramentas.

Desafio da aula

Agora é a sua vez.

Crie um programa com o seguinte menu:

```
=== MENU ===
```

```
1 - Ver meu nome  
2 - Ver minha idade  
3 - Ver minha profissão  
4 - Sair
```

Utilize:

```
switch  
case  
break  
default
```

Depois tente melhorar o programa fazendo o menu aparecer novamente até o utilizador escolher a opção **4**.

Se conseguir, tente adicionar mais uma opção:



5 - Fazer uma conta

E dentro dessa opção faça uma pequena calculadora.

O que aprendemos nesta aula

Nesta aula aprendemos:

- O que é `switch` ;
- O que é `case` ;
- Para que serve `break` ;
- Como utilizar `default` ;
- Como criar menus;
- Como utilizar `switch` com números;
- Como utilizar `switch` com `char` ;
- Como combinar `switch` com `if` ;
- Como combinar `switch` com loops;
- Como criar uma calculadora simples;
- Como criar um pequeno sistema de conta.

A ideia não é decorar:

```
switch
case
break
default
```

A ideia é entender quando cada um pode ser útil.

Até agora, já temos várias ferramentas na nossa caixa:

```
Variáveis
Arrays
Operadores
if / else
for
while
switch / case
Funções
```

Agora começa uma parte ainda mais interessante: aprender a organizar tudo isso para construir programas maiores.